# On-Premises LLM Assistance for Embedded Flight Software: Measured Sizing, Traceability Recovery and Execution-Verified Test Generation

Djoud El Hak Belkhir
M2 Data, Knowledge and Hybrid AI
Université Paris-Saclay, France

***Abstract*—Teams that build embedded radio and flight software often cannot send source code to a hosted model, yet they want the same assistance for coding, refactoring, documentation and testing. We present an open, fully on-premises platform that serves open-weight code models, connects them to a team's code, tests, requirements and CI through the Model Context Protocol (MCP), and sizes the hardware from measurements. We study three questions on NASA's F´ flight-software framework with two 2025 open-weight models (Qwen3-Coder-30B-A3B, a mixture-of-experts model, and the dense Devstral-Small-2-24B) served on two Tesla T4 GPUs. (i) A roofline model calibrated per engine and architecture class predicts single-stream decode speed within $\pm$11 % leave-one-model-out and the chat capacity of one llama.cpp server within 7 %. (ii) Ranking the 3,261 test cases of F´ for each of 151 requirements, reciprocal-rank fusion of BM25 and dense retrieval inside a module-graph prior recovers requirement-to-test links with a mean average precision of 0.609, while an LLM reranker lowers it. (iii) An agent that may write only test code, and whose tests must pass a policy check, compile and pass before being committed for review, closes 18 of 28 traceability gaps with each model; it closed 11 and 8 before a failure taxonomy pointed to general prompt and tooling fixes (exact McNemar $p = 0.016$ and $0.013$). Its tests kill 66 % and 52 % of seeded mutants. All code, data and notebooks are released.**

***Index Terms*—large language models, on-premises deployment, capacity planning, requirements traceability, unit test generation, flight software, Model Context Protocol, mutation testing**

## I. Introduction

Code assistants built on large language models (LLMs) are now common in software teams, but the default deployment, a hosted model that receives the code, is ruled out for many embedded and defence projects by export control, contractual or security constraints. Open-weight code models make on-premises use possible. Three practical questions then decide whether such a deployment is worth it, and they are rarely answered together. *How much hardware* does a team need, given that vendor peak numbers do not predict engine behaviour? *How is the model grounded* in the team's own artefacts (requirements, tests, design documents) rather than in its training data? And *how can its output be trusted* in a domain where a plausible but wrong test is worse than no test?

We answer them on a single, reproducible case: NASA's open-source F´ framework [1] (v4.4.1), whose design documents define requirements and whose unit tests cite them with `REQUIREMENT("ID")` tags. Our contributions are:

- an open on-premises platform (Fig. 1): engine-agnostic model serving, a typed knowledge graph of code, tests, requirements and documents exposed through an MCP server [2], CI access through Jenkins' MCP plugin, and an air-gap smoke test run in CI;
- a sizing method that calibrates a roofline model [3] separately for dense and mixture-of-experts (MoE) models and for each engine, validated leave-one-model-out and against load tests;
- a traceability-recovery benchmark on F´ (151 requirements $\times$ 3,261 tests, tags masked) comparing lexical, dense, graph-constrained and LLM-reranked retrieval with bootstrap confidence intervals;
- a bounded-authority test-generation agent whose every output is gated by build, test and policy checks, and a taxonomy of its 121 failed attempts that pointed to general fixes and a significant improvement.

## II. Related Work

*LLM-based unit test generation.* Schäfer et al. [4] generate JavaScript tests from API documentation and repair failing ones; CodaMosa [5] uses an LLM to escape coverage plateaus of search-based testing [6]; Meta's TestGen-LLM [7] keeps a generated test only if it builds, passes reliably and increases coverage, a filtering principle we adopt and extend with mutation analysis [8] and requirement tags. Agentic systems such as SWE-agent [9] act on repositories through tool interfaces; our agent instead restricts the model to test code and keeps every other action deterministic. Public benchmarks (HumanEval [10], SWE-bench [11]) are Python-centred and exposed to training-set contamination, which motivates the small, execution-scored embedded C suite of Section VI-C.

*Traceability recovery.* Information-retrieval approaches to trace links date back to Antoniol et al. [12] and are surveyed by Borg et al. [13]; LLM prompting for trace links was studied by Rodriguez et al. [14]. We combine BM25 [15],

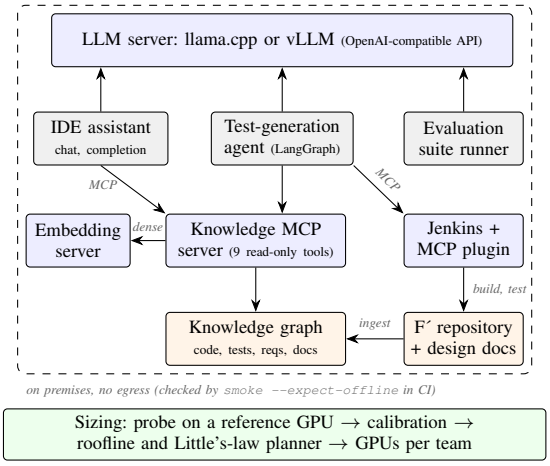



Fig. 1. Platform architecture. Clients reach models through an OpenAI-compatible API and engineering knowledge through MCP; the knowledge server's dense search calls the local embedding server. Everything runs inside the site boundary; sizing is an offline pipeline.

dense embeddings [16] and reciprocal-rank fusion [17] with a prior from the module graph, and use the retrieval both as a benchmark and as the context source of a retrieval-augmented agent [18].

*Serving and sizing.* vLLM [19] batches requests with paged KV-cache memory; llama.cpp [20] runs quantized GGUF models on modest GPUs. Quantization (e.g., AWQ [21]) and attention kernels (FlashAttention-2 [22]) change achievable throughput by GPU generation, which is why we calibrate on measurements rather than on datasheets.

## III. Platform

*Knowledge graph.* An ingester parses F′ with tree-sitter and the design documents' requirement tables into a typed graph: 1,250 files (206,656 lines of code), 5,499 functions, 3,261 test cases, 560 requirements from 83 documents, 21,681 resolved call edges (including the F′ autocoder conventions, e.g. a test calling `sendCmd_X` exercises `X_cmdHandler`) and 481 VERIFIES edges taken only from explicit tags. Identifiers derive from source locations, so the same commit always yields a byte-identical snapshot; it builds in about 3 s on a laptop CPU. The graph doubles as an audit: 288 requirements specified as verified by test have no tagged test, 12 cited IDs are defined nowhere and 12 are cited by another component's tests.

*MCP server.* Nine read-only tools serve IDE assistants and the agent alike: text and semantic search, symbol lookup, callers and callees, requirement lookup and tracing (`trace_requirement` ranks candidate tests and related code), coverage gaps per module, and module listings and overviews. They run over stdio or HTTP, from an in-memory snapshot or Neo4j.

*Deployment.* Container profiles cover CPU, single-GPU llama.cpp and multi-GPU vLLM. Model servers require API keys and bind to localhost; an offline overlay moves all services to an internal network and runs a smoke test that fails if any outbound connection succeeds, on every CI run.

TABLE I
PROBE ON 2×T4, 4-BIT WEIGHTS, SINGLE STREAM

| Model | Engine | Decode tok/s | Prefill tok/s | 4 streams tok/s |
|---|---|---|---|---|
| Qwen3-Coder-30B-A3B (MoE) | llama.cpp | 81.9 | 1,445 | 151 |
| Qwen3-Coder-30B-A3B (MoE) | vLLM AWQ | 101.0 | 283 | 245 |
| Qwen3.6-35B-A3B (MoE) | llama.cpp | 60.9 | 1,326 | 107 |
| Devstral-Small-2-24B (dense) | llama.cpp | 15.9 | 649 | 28 |
| Qwen3.8-27B (dense) | llama.cpp | 13.9 | 550 | 29 |

## IV. Sizing Model

Memory is the weight size (measured artefact, or parameters × bits per weight) plus, per sequence, the KV cache $2\,L_{\mathrm{attn}} H_{kv} d_h b\, n$ for $n$ tokens (and recurrent state for linear-attention layers). Decode is bandwidth bound: a step at batch $B$ reads the touched weights and every sequence's cache,

$$t_{\mathrm{dec}}(B) = \frac{W_{\mathrm{dense}} + W_{\mathrm{exp}}\big(1-(1-k/E)^{B}\big) + B\,c\,s_{kv}}{\eta_d\,\beta\,g}, \tag{1}$$

where an MoE layer routes each token to $k$ of $E$ experts, $c$ is the average context, $s_{kv}$ the cache bytes per token, $\beta$ the peak bandwidth and $g$ the tensor-parallel scale (1 for llama.cpp's layer split, whose GPUs work in sequence). Prefill is compute bound at $2P_{\mathrm{act}}$ FLOPs per token plus attention, at a fraction $\eta_p$ of peak FP16. The efficiencies $\eta_d, \eta_p$ are the only fitted parameters, estimated per engine and separately for dense and MoE models because a single shared value mispredicted MoE prefill by up to $+239\,\%$. Capacity then follows from Little's law [23]: with request classes of rate $\lambda_i$ and service time $T_i$, the concurrent sequences $C = \sum_i \lambda_i T_i$ must fit in $B^*$, the largest batch whose per-sequence decode speed meets the service-level objective (SLO), after the replica's prefill time $U$ is deducted: $R \geq h\,(C/B^* + U)$ replicas for headroom $h$.

## V. Experimental Setup

All GPU measurements ran in Kaggle batch notebooks on two NVIDIA Tesla T4 GPUs (16 GB, Turing, SM 7.5). We compare Qwen3-Coder-30B-A3B-Instruct [24] (MoE, 30.5 B parameters, 3.3 B active, 8 of 128 experts) and Devstral-Small-2-24B [25] (dense, 24.0 B), both released in 2025 under Apache 2.0. Both run on llama.cpp (build b11953, Q4_K_M weights, layer split, 4 slots of 16k tokens); Qwen3-Coder also runs on vLLM 0.32 (AWQ, tensor parallel 2). A probe run measured five models; benchmark run 2 (3.4 h) produced the traceability, evaluation and agent-v1 results and run 3 (2.8 h) the serving sweeps and agent v2. Table I gives the single-stream baseline. On this bandwidth-bound GPU the MoE model decodes about five times faster than the dense one, and vLLM's AWQ prefill (no Marlin kernels, no FlashAttention-2 on SM 7.5) is five times slower than llama.cpp's.

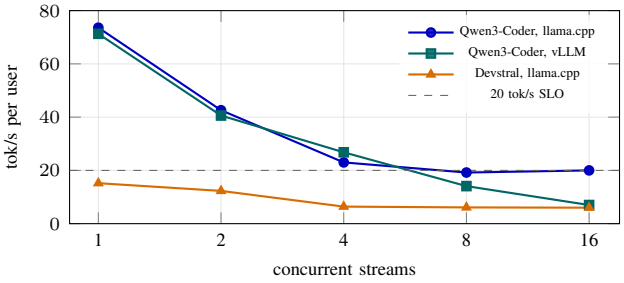



Fig. 2. Decode speed each user sees in the chat profile (median), benchmark run 3. The dashed line is the 20 tok/s SLO used for sizing.

## VI. Results

### A. Serving capacity and sizing accuracy

Fig. 2 shows the IDE-chat profile (about 2.5k-token prompts, 400-token answers); every concurrency level uses new prompts so that no level is served from a cache warmed by the previous one. At a 20 tok/s per-user SLO, one 2×T4 server holds about four concurrent chat users with either engine. llama.cpp stops scaling at its four slots: past them, requests queue and the median time to first token grows from 5.7 s to 71 s at 16 streams. vLLM keeps batching (98 tok/s total at 16 streams) at 7 tok/s per user. The ranking flips for agent steps (about 9k-token prompts with a shared prefix): llama.cpp serves 0.29–0.44 requests/s against 0.10–0.14 for vLLM, whose long-prompt prefill is the bottleneck on Turing. Devstral stays below the SLO even alone (15 tok/s).

Table II validates the sizing model. Fitted on the probe, the llama.cpp efficiencies are $\eta_d = 0.771/0.590$ and $\eta_p = 0.508/0.202$ (dense / MoE); vLLM reaches $\eta_d = 0.374$ and $\eta_p = 0.030$ on this GPU. Decode is predicted within $\pm 11\,\%$ leave-one-model-out and within 4 % on run 3, whose contexts the calibration never saw. MoE prefill is not predictable from one other MoE model with a different attention design (Qwen3.6 has linear-attention layers). The planner's chat capacity for one llama.cpp server is within 7 % of the load test; on vLLM it is conservative by half, because its prefill efficiency was fitted on a 13k-token prompt and Turing prefill is much slower on long prompts. Applied to larger GPUs, the same planner gives, for 100 engineers using chat and agents, 6 L4 or 2 L40S or 1 H100 for Qwen3-Coder; such tables hold only to first order until the probe is re-run on the candidate GPU, which the tooling is designed for.

### B. Requirement-to-test traceability recovery

The 481 tags link 151 requirements to their tests. We mask tags and IDs and let each method rank all 3,261 test cases for every requirement; ties are broken randomly and averaged. Dense retrieval uses Qwen3-Embedding-0.6B with task instructions. The graph prior ranks tests of the requirement's module first, then tests that call into it. Table III reports means with 95 % bootstrap intervals [26]. Lexical and dense retrieval are equally good alone but fail on different requirements: their fusion inside the graph prior is best (MAP 0.609), +0.04 over graph-constrained BM25. Pointwise LLM reranking of the top ten lowers precision at the top (Recall@1 from 0.350 to 0.297): the judge scores pairs without the module context and produces many ties, at a cost of ten model calls per requirement.

TABLE II
Sizing model validation (llama.cpp unless noted)

| Prediction | Measured | Predicted | Error |
|---|---|---|---|
| *Leave-one-model-out, decode tok/s* | | | |
| Qwen3.8-27B (dense) | 13.9 | 13.7 | −1.5 % |
| Devstral-Small-2-24B (dense) | 15.9 | 16.1 | +1.5 % |
| Qwen3-Coder-30B-A3B (MoE) | 81.9 | 90.6 | +10.6 % |
| Qwen3.6-35B-A3B (MoE) | 60.9 | 55.1 | −9.6 % |
| *Independent run 3, decode tok/s at 2.8k context* | | | |
| Qwen3-Coder | 73.6 | 76.7 | +4 % |
| Devstral | 15.2 | 15.6 | +3 % |
| *Chat capacity, requests/s at 20 tok/s per user* | | | |
| Qwen3-Coder, llama.cpp | 0.174 | 0.187 | +7 % |
| Qwen3-Coder, vLLM | 0.185 | 0.090 | −51 % |

TABLE III
Traceability recovery on F′ (151 requirements, 3,261 candidates)

| Method | MAP [95 % CI] | R@1 | R@10 |
|---|---|---|---|
| Graph prior only | 0.249 [0.220, 0.278] | 0.073 | 0.550 |
| Dense (Qwen3-Embedding-0.6B) | 0.479 [0.428, 0.531] | 0.261 | 0.713 |
| BM25 | 0.488 [0.427, 0.548] | 0.297 | 0.677 |
| Graph + BM25 | 0.569 [0.513, 0.626] | 0.330 | 0.781 |
| Graph + dense | 0.573 [0.521, 0.619] | 0.303 | 0.817 |
| **Graph + RRF(BM25, dense)** | **0.609 [0.553, 0.660]** | **0.350** | **0.821** |
| + LLM rerank of top 10 | 0.584 [0.535, 0.633] | 0.297 | 0.822 |

### C. Embedded C evaluation suite

To choose a model for the assistant use cases we wrote 27 C99 tasks in the style of radio and flight software (CCSDS headers, COBS framing, CRCs, Hamming and convolutional codes, Q15 arithmetic, wrap-around-safe timers) in four categories, all scored by execution: *generate* (hidden tests under `-Wall -Wextra -Wpedantic`, no heap or stdio), *test generation* (the tests must pass on the correct implementation and fail on every one of 4–6 seeded mutants), *refactor* (tests still pass and maximum cyclomatic complexity strictly decreases) and *document* (code tokens unchanged and complete Doxygen tags). The suite validates itself: references pass, generated stubs fail, reference tests kill every mutant, and prompts never contain hidden material. Table IV gives one sample per task at temperature 0.2. Qwen3-Coder passes 15 tasks against 10, with a four times lower median latency; on the same tasks 7 are passed only by Qwen3-Coder and 2 only by Devstral (exact McNemar [27] $p = 0.18$), a direction rather than a significant difference. Re-scoring every failure from the stored outputs shows the failures are the models': 8 of 12 generated test programs fail on the *correct* implementation because hand-computed expected values are wrong, both models mishandle stated bit-level conventions

TABLE IV
EMBEDDED C SUITE: TASKS PASSED (ONE SAMPLE, PASS@1)

| Model | Gen. /12 | Tests /6 | Refact. /5 | Doc. /4 | Total /27 | Median latency |
|---|---|---|---|---|---|---|
| Qwen3-Coder | **9** | 0 | 2 | **4** | **15** | **21 s** |
| Devstral | 5 | 1 | 2 | 2 | 10 | 85 s |

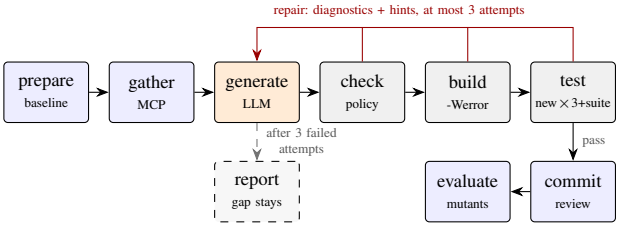



Fig. 3. The agent's graph. Only *generate* calls the LLM; every other node is deterministic and takes its arguments from the requirement, never from model output. Red edges are repairs; the dashed edge ends the run without a commit.

(COBS blocks, Hamming bit layout), and failed refactorings change behaviour.

### D. Test generation for traceability gaps

*Agent design.* The agent (Fig. 3) is a LangGraph [28] state machine. The model writes only new `Tester` methods and `TEST()` cases in two fenced blocks; declarations, insertion points and includes are derived by a patcher. Before building, a policy rejects changes outside the component's three test files, reused names, missing assertions, unsafe calls (`system`, `fork`, threads, inline assembly) and any requirement tag other than the target's. A test is accepted only if the F′ unit tests build with `-Werror`, the new test passes three runs (F′ tests use random seeds) and the full suite still passes; it is then committed on a review branch with an AI-use disclosure, as F′'s contribution policy asks. Context is deterministic and budgeted at 32,000 characters, so that a 16k-token slot holds the task, one previous answer, its diagnostics and the new answer. Repairs (at most three attempts) receive condensed compiler errors with did-you-mean hints, test output, and for F′ assertions the asserted source line. Fault detection is measured after the fact, never shown to the model: the new test alone runs against up to four mutants of the lines it executes (inverted conditions, swapped success statuses, deleted port calls).

*Targets.* We select the 28 gaps of four components (`Svc/ComStub`, `Svc/ComRetry`, `Svc/PrmDb`, `Svc/FprimeRouter`) that have the standard test harness, build and pass at baseline, and show no ID mismatch in the audit.

*From v1 to v2.* We classify every failed attempt by its first matching diagnostic rule (Fig. 4). In v1, 65 of 121 failed attempts were dispatch calls on the wrong object or on passive components: the prompt described the active-component dispatch rule without stating which kind the component under test was (three of four are passive), and the did-you-mean hints

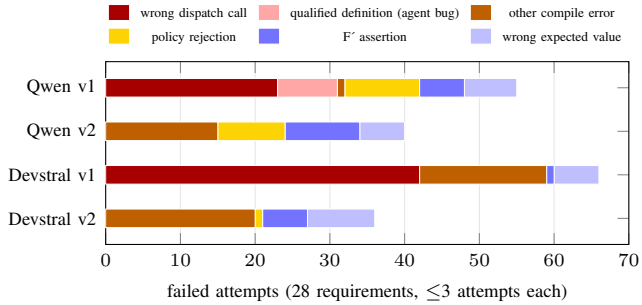



Fig. 4. Failed attempts by primary cause, agent v1 (run 2) and v2 (run 3), same 28 requirements, models and context budget.

TABLE V
TEST-GENERATION AGENT ON 28 F′ GAPS (V1 → V2)

| Model | Verified /28 | 1st answer compiles | Attempts if verified | Mutants killed | Median time |
|---|---|---|---|---|---|
| Qwen3-Coder | 11 → **18** | 52 → 81 % | 1.56 | 42/64 (66 %) | 43 s |
| Devstral | 8 → **18** | 15 → 67 % | 1.33 | 35/67 (52 %) | 94 s |

offered a function that F′ v4 no longer has. Eight more came from a patcher bug on namespace-qualified definitions. Agent v2 states the component kind and its exact dispatch call in the task, fixes the patcher, reuses a code block that a repair answer leaves out, adds the asserted line to assertion failures and lists the `-Werror` rules. These are general rules, not per-requirement fixes. Both causes fell to zero (Fig. 4).

*Results.* Table V summarises v2 and Table VI breaks it down by component; Fig. 5 shows one repaired run end to end. Verified requirements rose from 11 to 18 for Qwen3-Coder (none lost) and from 8 to 18 for Devstral (12 gained, 2 lost); per requirement, the exact McNemar test gives $p = 0.016$ and $p = 0.013$. Together the models verify 21 of the 28 gaps, 15 by both. The remaining failures are the models' own: invented API names (e.g. a non-existent enumerator of `Fw::ParamValid`), include paths, broken component preconditions and wrong expected values. Some counts rise in v2 because requirements that used to stop at the dispatch error now reach the build of their real content. An AI pre-review of the 36 verified tests judges 30 relevant and 6 partial (one case of a multi-case requirement), none irrelevant; these verdicts await confirmation by an engineer on the generated review sheets. After a batch, the agent can have Jenkins run the component test suites on its review branch through Jenkins' MCP plugin with a build-only service account; this path is exercised in CI with a scripted model, not in the Kaggle runs. Line-coverage gains are zero: the chosen components were already 92–98 % covered, which is why fault detection, not coverage, is the useful automatic signal here.

## VII. DISCUSSION AND THREATS TO VALIDITY

*What transfers.* Three findings are likely to hold beyond this setup: MoE models with about 3 B active parameters are the practical choice on bandwidth-limited GPUs; the right engine

```
SVC-COMRETRY-002: store Fw::Buffer and its context on
receiving buffer ownership through dataReturnIn
attempt 1, build: error: 'class ComCfg::FrameContext'
  has no member named 'frameType'
attempt 2, test failed:
  Assert: "Svc/ComRetry/ComRetry.cpp:85"
  hint: FW_ASSERT(this->m_bufferState ==
          Fw::Buffer::OwnershipState::NOT_OWNED);
attempt 3, passes 3 runs + suite (6 tests); excerpt:
  REQUIREMENT("SVC-COMRETRY-002");
  this->invoke_to_dataIn(0, buffer, context);
  this->invoke_to_dataReturnIn(0, buffer, context);
  this->invoke_to_comStatusIn(0, success);
  ASSERT_from_dataReturnOut_SIZE(1);
  ASSERT_from_dataReturnOut(0, buffer, context);
mutants: 2/4 killed; survivors: && -> || (line 42),
  deleted comStatusOut_out(...) (line 47)
```

Fig. 5. A real repaired run (Qwen3-Coder, run 3, 45 s, commit `574cb8d`). The second repair succeeds because the asserted line shows the precondition the test broke; the surviving mutants show what the test does not check (forwarding of the status).

TABLE VI
VERIFIED REQUIREMENTS PER COMPONENT, V1 → V2

| Component | Kind | Gaps | Qwen3-Coder | Devstral | Either (v2) |
|---|---|---|---|---|---|
| `Svc/ComStub` | passive | 7 | 4 → 7 | 3 → 7 | 7 |
| `Svc/ComRetry` | passive | 7 | 0 → 3 | 2 → 2 | 4 |
| `Svc/PrmDb` | active | 5 | 3 → 3 | 1 → 3 | 3 |
| `Svc/FprimeRouter` | passive | 9 | 4 → 5 | 2 → 6 | 7 |
| Total | | 28 | 11 → 18 | 8 → 18 | 21 |

depends on the traffic mix (interactive batching versus long agent prompts); and in this domain, gating generated tests by execution and mutation is necessary, since even the better model wrote wrong oracles for most of its tests in the C suite. The largest agent gain came from diagnosing failures, not from changing the model.

*Internal validity.* Sample sizes are small (27 tasks, 28 requirements) and each task was sampled once, so we report exact paired tests and intervals rather than rankings. Prompts, context and attempt budgets are identical across models. The serving confound we found in run 2 (repeated prompts served from cache) was fixed and re-measured; the superseded numbers are kept and labelled. *External validity.* All GPU numbers are from Turing GPUs; vLLM in particular lacks optimised kernels there, and its results must be re-measured on Ampere or Hopper. F´ is public and may be in the models' training data, which would favour both models equally; the C suite was written for this study. *Construct validity.* Relevance was pre-assessed by an LLM and is pending human review; mutation scores use a sample of four mutants per test.

## VIII. Conclusion

A small team can run useful code assistance entirely on premises today: two 16 GB GPUs serve four interactive users or a test-generation agent, a calibrated model sizes larger deployments within single-digit percent for the calibrated engine, graph-constrained hybrid retrieval recovers requirement-to-test links at MAP 0.61, and an agent with bounded authority closes two thirds of a component's traceability gaps with tests that compile, pass and catch most seeded faults, each committed for human review. Future work: resolve unknown identifiers to real enumerators and headers before repair, re-calibrate on Ampere and Hopper GPUs, and complete the human relevance review.

## Artifact Availability

Code, Kaggle notebooks, raw results and the generated tests (as patches on F´ v4.4.1) are available at https://github.com/djihad-elhak/onprem-genai-embedded.

## Acknowledgment

The author used an AI assistant for parts of the implementation and drafting; all measurements were produced by the released notebooks and checked against the stored records.

## References

[1] R. Bocchino, T. Canham, G. Watney, L. Reder, and J. Levison, "F Prime: An open-source framework for small-scale flight software systems," in *Proc. 32nd Annual AIAA/USU Conference on Small Satellites (SmallSat)*, Logan, UT, USA, 2018.

[2] Model Context Protocol, "Model Context Protocol specification," https://modelcontextprotocol.io/specification, 2025.

[3] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[4] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.

[5] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *Proc. IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023.

[6] G. Fraser and A. Arcuri, "EvoSuite: Automatic test suite generation for object-oriented software," in *Proc. 19th ACM SIGSOFT Symposium and 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, 2011, pp. 416–419.

[7] N. Alshahwan, J. Chheda, A. Finogenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, "Automated unit test improvement using large language models at Meta," in *Companion Proc. 32nd ACM International Conference on the Foundations of Software Engineering (FSE), Industry Track*, 2024.

[8] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[9] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[10] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[11] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[12] G. Antoniol, G. Canfora, G. Casazza, A. De Lucia, and E. Merlo, "Recovering traceability links between code and documentation," *IEEE Transactions on Software Engineering*, vol. 28, no. 10, pp. 970–983, 2002.

[13] M. Borg, P. Runeson, and A. Ardö, "Recovering from a decade: A systematic mapping of information retrieval approaches to software traceability," *Empirical Software Engineering*, vol. 19, no. 6, pp. 1565–1616, 2014.

[14] A. D. Rodriguez, K. R. Dearstyne, and J. Cleland-Huang, "Prompts matter: Insights and strategies for prompt engineering in automated software traceability," arXiv:2308.00229, 2023.

[15] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[16] Y. Zhang *et al.*, "Qwen3 Embedding: Advancing text embedding and reranking through foundation models," arXiv:2506.05176, 2025.
[17] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
[18] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
[19] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
[20] G. Gerganov and llama.cpp contributors, "llama.cpp: LLM inference in C/C++," https://github.com/ggml-org/llama.cpp, build b11953, 2026.
[21] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. Machine Learning and Systems (MLSys)*, 2024.
[22] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.
[23] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.
[24] Qwen Team, "Qwen3 technical report," arXiv:2505.09388, 2025.
[25] Mistral AI, "Introducing Devstral 2 and Mistral Vibe CLI," https://mistral.ai/news/devstral-2-vibe-cli, Dec. 2025.
[26] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.
[27] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
[28] LangChain, "LangGraph: Build resilient language agents as graphs," https://github.com/langchain-ai/langgraph, 2025.